LSI1 Tassadit Ouzia, Oumaima El Khadraoui, Juliette Jean, Nivine Lahouar

PROJET FINAL

CAMPUS EVENT

Présentation du système et du besoin

Une université organise régulièrement des conférences, ateliers, forums entreprises, événements associatifs, journées portes ouvertes et rencontres professionnelles. Aujourd'hui, la gestion des événements et des inscriptions repose sur plusieurs fichiers et formulaires indépendants, ce qui entraîne des informations dispersées et parfois incohérentes, des risques de double inscription, une difficulté à connaître le nombre de places restantes, un suivi incomplet des participants et une vision limitée pour les organisateurs et l'administration.
L'objectif métier est de centraliser les événements, les étudiants et les inscriptions dans un même système d'information nommé CAMPUS EVENT. Un étudiant consulte la liste des événements, en choisit un et demande à s'y inscrire. Le système vérifie les informations reçues, l'existence de l'étudiant et de l'événement, la disponibilité des places et l'absence de double inscription. Si toutes les conditions sont réunies, une inscription est créée avec le statut pending ; l'administration peut ensuite la faire passer à accepted ou rejected ; si l'événement est complet, la demande est placée en liste d'attente (waitlisted).
Ce rapport présente l'analyse du système d'information, l'architecture 3-tiers retenue, la conception et le développement de l'API REST avec Flask, la campagne de tests Postman et la modélisation UML.

1. Analyser le système d'information

a. Identifier les acteurs

Acteur	Rôle	Informations consultées	Informations créées / modifiées
Étudiant	Consulte les événements et demande à s'y inscrire	liste des événements, dates, places disponibles, statut de ses inscriptions	demandes d'inscription
Organisateur	prépare les événements et suit leur déroulement	informations des événements, nombres d'inscrits, places restantes	Informations des événements : titre, date, lieu et capacité
Administration	Gère les demandes d'inscription, assure leur suivi et pilote	Liste des étudiants, événements et inscriptions en	Statut des inscriptions (accepted ou

	l’activité	attente, statistiques (nombre d’événements et d’inscriptions, taux de remplissage)	rejected)

b. Identifier les processus métier

Processus	Objectif métier	Étapes principales	Acteur(s) responsable(s)
S’inscrire à un événement	Réserver une place sans erreur	1. Consulter les événements 2. Choisir un événement 3. Envoyer la demande 4. Le système vérifie (étudiant, événement, places, double inscription) 5. L’inscription est créée en *pending*	Étudiant
Créer un événement	Proposer un nouvel événement	1. Remplir titre, date, capacité 2. Enregistrer 3. L’événement apparaît dans la liste	Organisateur
Valider une inscription	Accepter ou refuser les demandes	1. Consulter les inscriptions *pending* 2. Décider 3. Passer le statut à *accepted* ou *rejected*	Administration
Suivre les statistiques	Piloter l’activité	1. Consulter les inscriptions 2. Calculer le	Administration

		remplissage 3. Analyser	

c. Identifier les informations

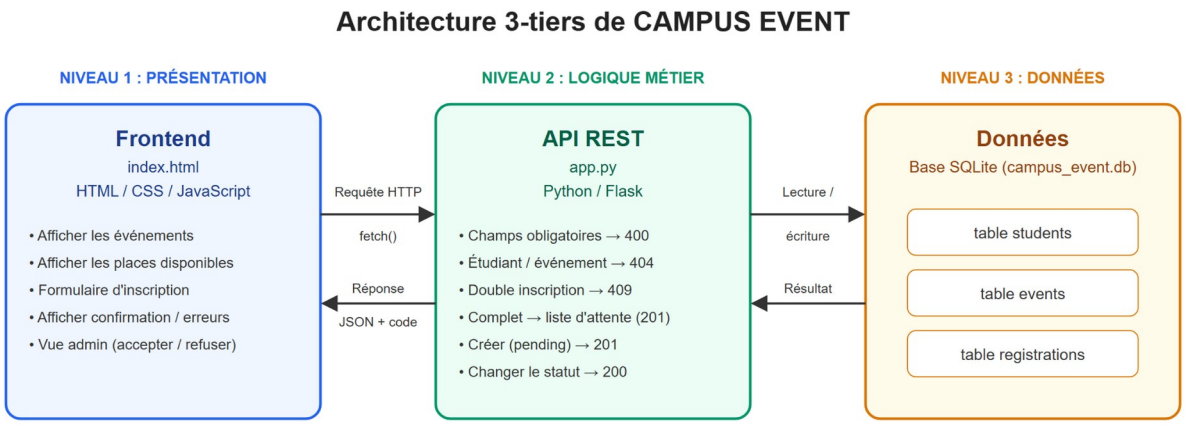
Catégorie	Informations principales	Qui les crée ?	Qui les consulte ?
Étudiants	id, nom, email, filière	Administration	Organisateur, Administration
Événements	id, titre, date, lieu, capacité, places disponibles	Organisateur	Étudiant, Organisateur, Administration
Inscriptions	id, student_id, event_id, statut	Étudiant (la demande), Administration (le statut)	Étudiant, Organisateur, Administration
Pilotage / statistiques	Nombre d'inscrits, taux de remplissage, nombre d'événements	Le système (calcul automatique)	Administration

d. Représenter les flux d'information

N°	Émetteur	Information envoyée	Destinataire
1	Étudiant	Demande d'inscription (student_id, event_id)	CAMPUS EVENT
2	CAMPUS EVENT	Confirmation ou message d'erreur	Étudiant
3	Organisateur	Nouvel événement (titre, date, capacité)	CAMPUS EVENT
4	Administration	Décision sur l'inscription (accepted /	CAMPUS EVENT

		rejected)	
5	CAMPUS EVENT	Statistiques d’inscription	Administration

2**. Proposer l’architecture du système**



Niveau	**Rôle**	**Traitements / responsabilités**	**Technologie proposée**
Présentation	permettre aux étudiants d’interagir avec le système	afficher les événements, saisir une demande d’inscription et afficher les réponses du serveur	HTML, CSS et JavaScript
Logique métier	traiter les demandes et appliquer les règles métier	vérifier l’existence de l’étudiant et de l’événement, les places et les doublons. Créer les inscriptions et gérer leur statut	Python et Flask
Données	Conserver les informations nécessaires au fonctionnement du système	stocker les étudiants, les événements et les inscriptions. Permettre leur consultation et leur	Base SQLite (fichier campus_event.db), remplie au premier lancement à partir des listes Python

		mise à jour par le backend	

3. Concevoir l'API REST

a. Ressources

Ressource métier	Nom REST proposé	Exemple de données importantes
Étudiant	/students	id, name, email, filière
Événement	/events	id, title, date, location, capacity
Inscription	/registrations	id, student_id, event_id, status
Autre ressource éventuelle	Non nécessaire	Les statistiques peuvent être calculées à partir des événements et inscriptions

b. Endpoints

Besoin métier	Méthode HTTP	Endpoint	Code succès attendu
Consulter tous les événements	GET	/events	200
Consulter un événement précis	GET	/events/<id>	200
Consulter les étudiants	GET	/students	200
Créer une	POST	/registrations	201

inscription			
Consulter les inscriptions	GET	/registrations	200
Consulter une inscription	GET	/registrations/<id>	200
Modifier le statut d'une inscription	PATCH	/registrations/<id>	200

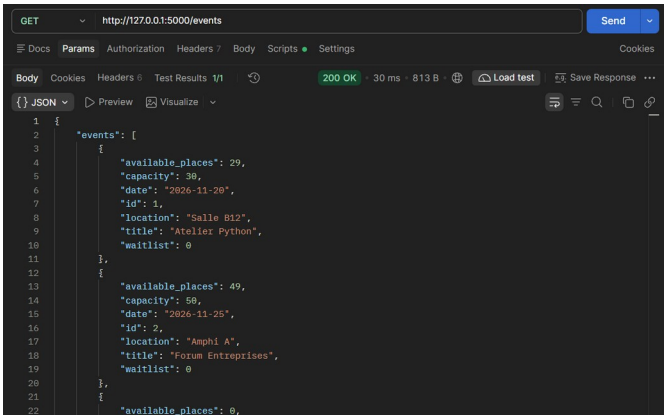
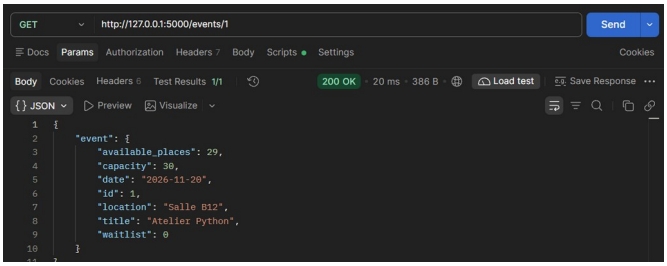
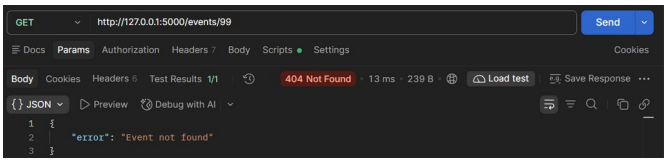
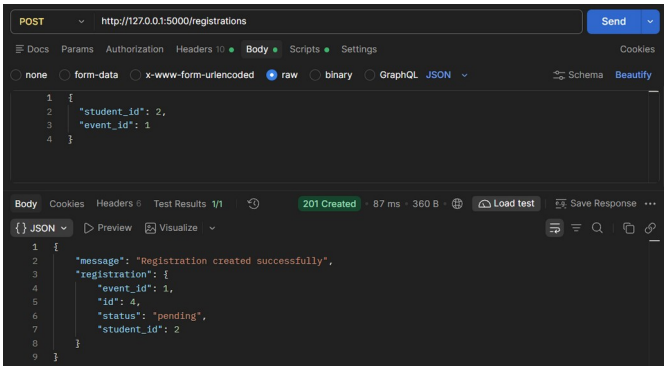
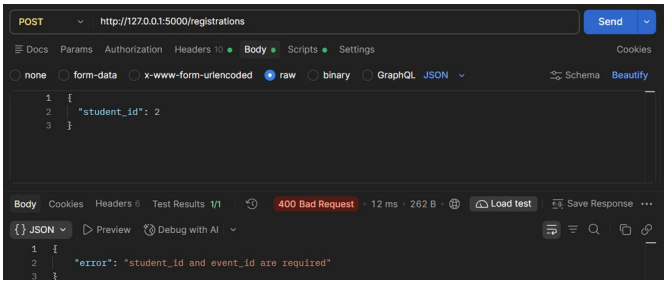
4. Développer l'API avec Flask

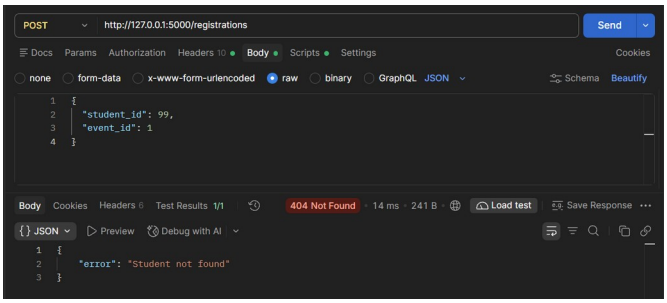
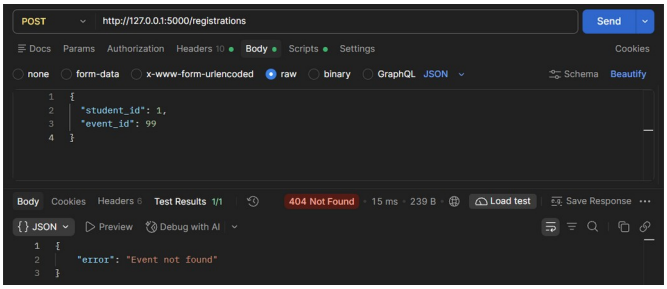
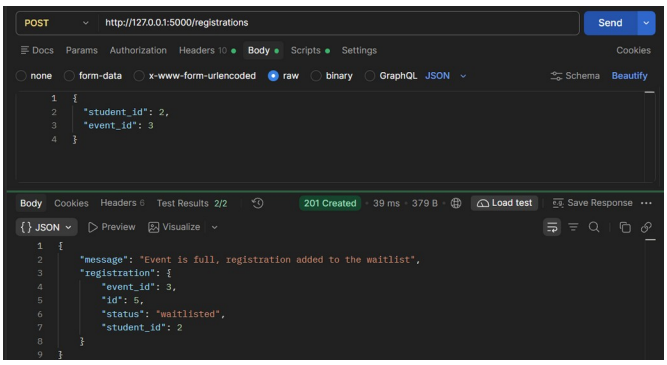
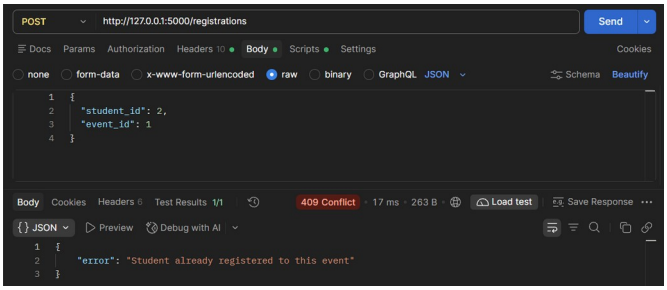
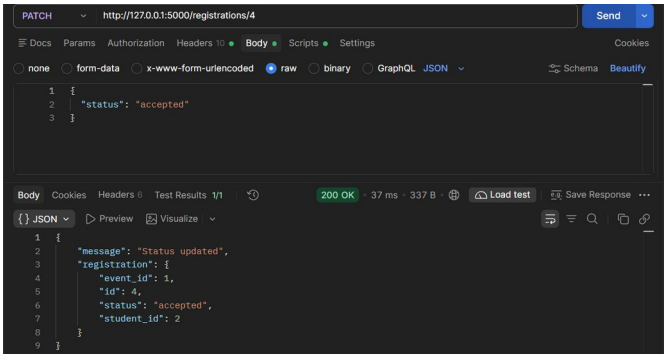
LE FICHIER app.py

Règles métier appliquées par l'API, dans cet ordre de vérification :

Règle	**Comportement**	**Code HTTP**
Champs obligatoires absents (student_id, event_id)	Refuser la demande	400 Bad Request
Étudiant inexistant	Refuser la demande	404 Not Found
Événement inexistant	Refuser la demande	404 Not Found
Double inscription (même étudiant, même événement)	Refuser la demande	409 Conflict
Événement complet (inscriptions pending ou accepted ≥ capacité)	Placer l'inscription en liste d'attente (statut waitlisted)	201 Created
Création réussie	Créer l'inscription avec le statut pending	201 Created
Modification du statut réussie	Mettre à jour le statut (pending, accepted, rejected ou waitlisted)	200 OK
Statut invalide lors de la modification	Refuser la demande	400 Bad Request
Inscription inexistante lors de la modification	Refuser la demande	404 Not Found
Accepter une inscription en liste d'attente alors que l'événement est toujours complet	Refuser la demande	409 Conflict

5. Tester l’API avec Postman

Test	Résultat attendu	Résultat obtenu	Capture / commentaire
GET /events	200	200 OK (Renvoie la liste des événements)	
GET événement existant	200	200 OK (Détails de l’événement)	
GET événement inexistant	404	404 Not Found	
POST inscription valide	201	201 Created (Statut initial pending)	
POST JSON incomplet	400	400 Bad Request	

POST étudiant inexistant	404	404 Not Found	
POST événement inexistant	404	404 Not Found	
POST événement complet	201	201 Created (statut waitlisted)	
POST double inscription	409	409 Conflict	
PATCH statut	200	200 OK (Mise à jour du statut en accepted)	

6. Modélisation UML

Diagramme de cas d’utilisation

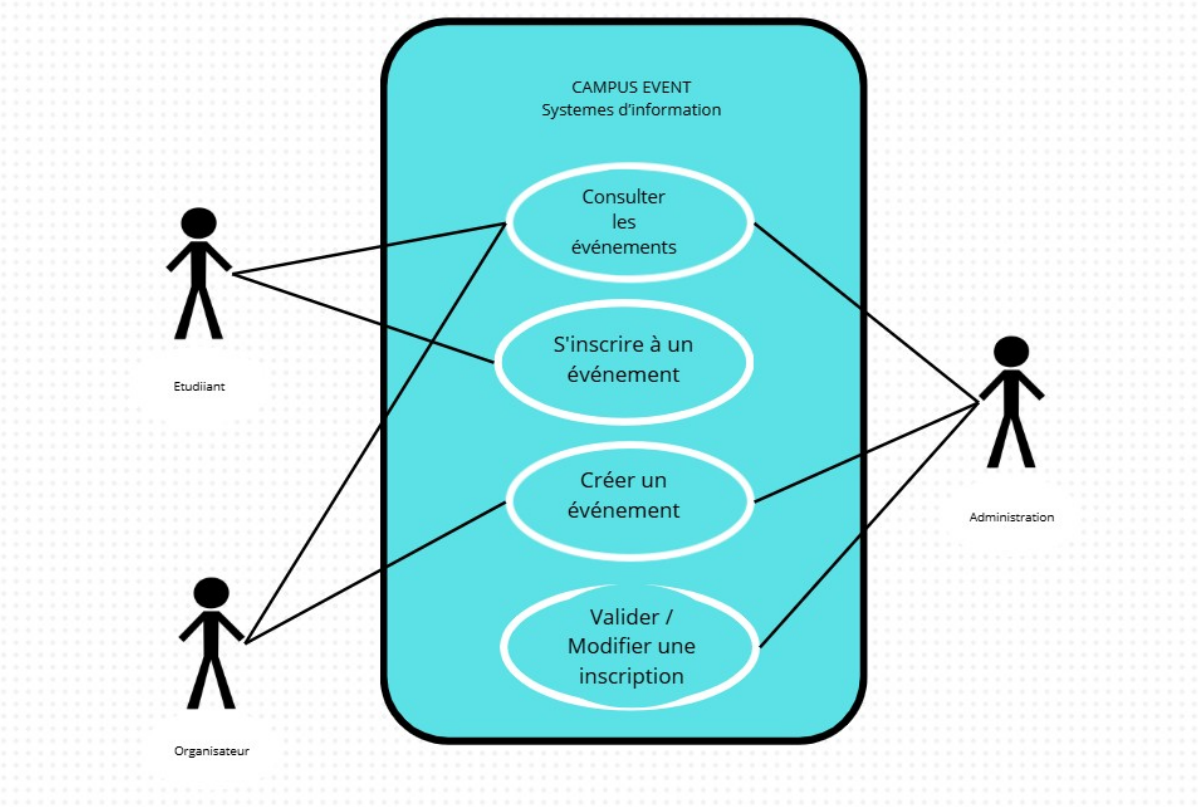


Diagramme d’Activité (Processus d’Inscription)

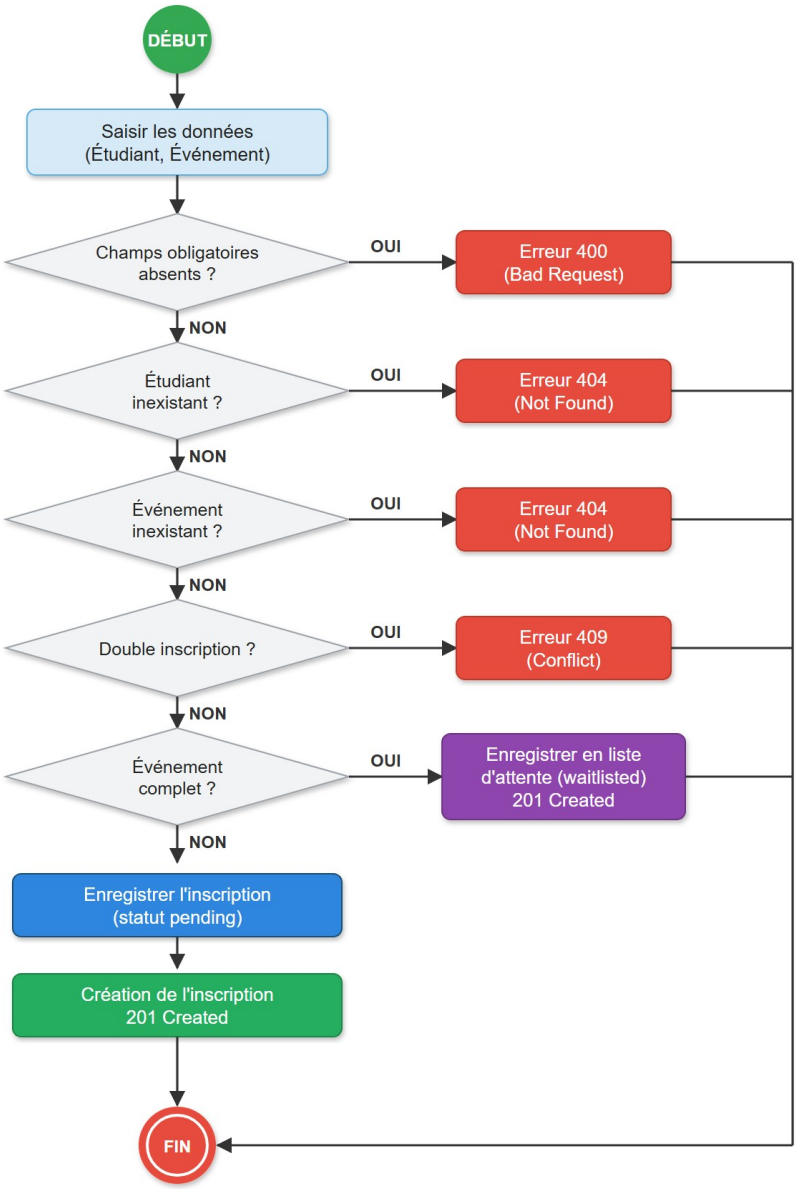


Diagramme de séquence (inscription à un événement):

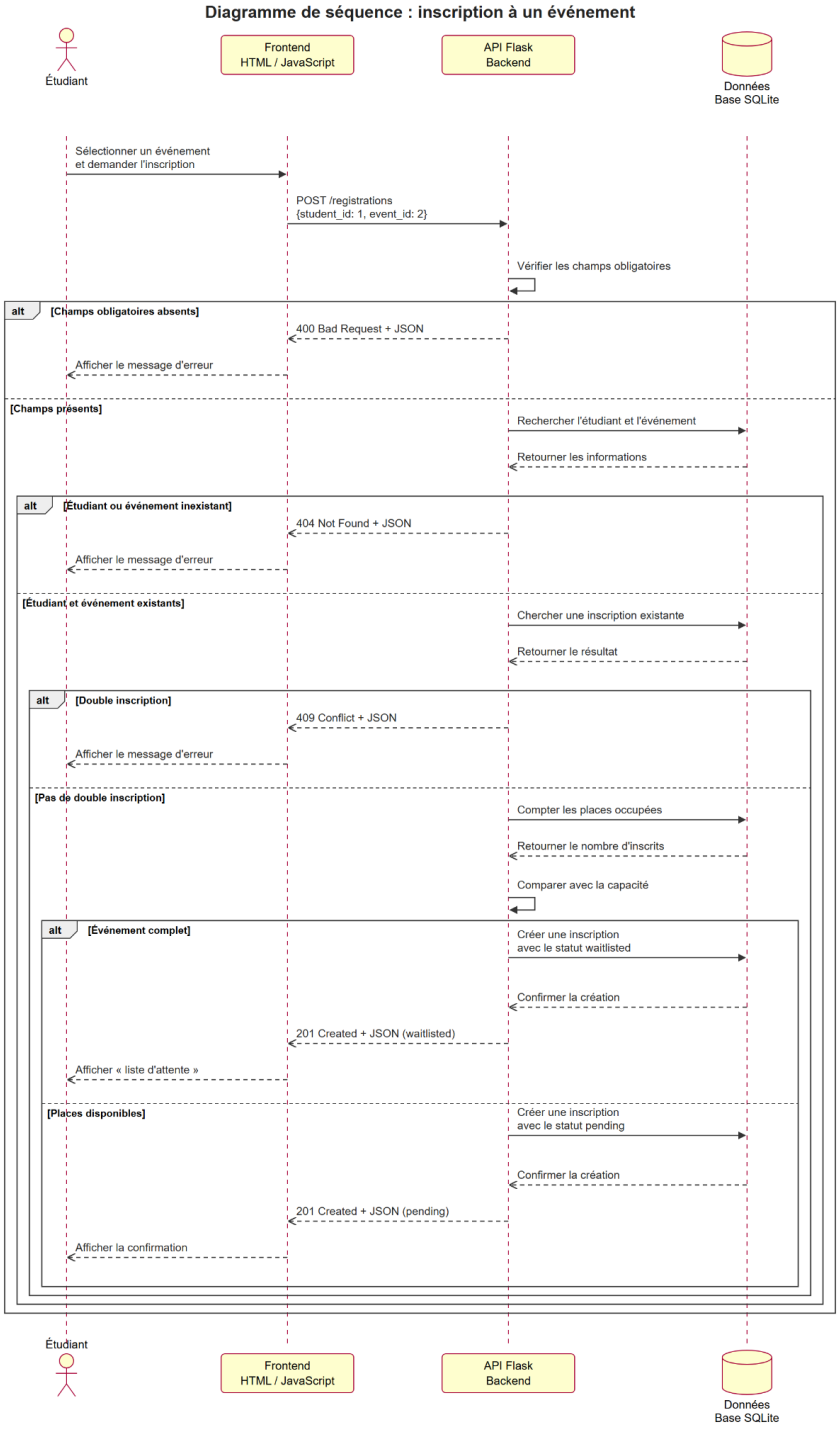
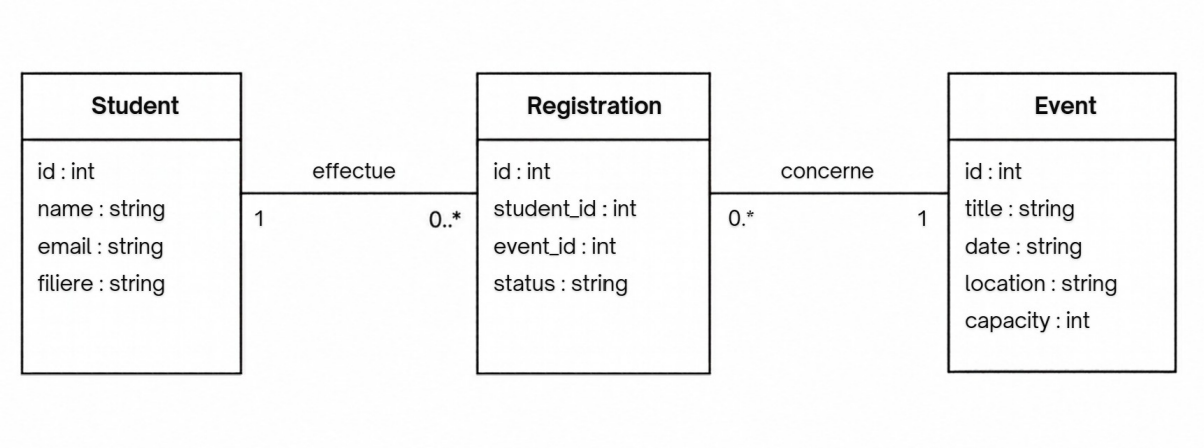


Diagramme de classes :



Difficultés rencontrées et choix réalisés

Cohérence entre les parties : les acteurs, les processus, les flux et les quatre diagrammes UML décrivent le même système. Nous avons attribué la validation des inscriptions (accepted / rejected) à l'administration et la création des événements à l'organisateur, puis aligné tous les tableaux et diagrammes sur ce choix.

Ordre des vérifications : l'API contrôle successivement les champs obligatoires (400), l'existence de l'étudiant puis de l'événement (404), puis la double inscription (409). Si l'événement est complet, l'inscription n'est pas refusée : elle est placée en liste d'attente (201, statut waitlisted). Le même ordre est utilisé dans le diagramme d'activité, le diagramme de séquence et le code.

Choix techniques : au départ, les données étaient dans des listes Python, comme demandé dans le sujet. Le problème, c'est qu'elles revenaient à zéro à chaque redémarrage du serveur. Nous avons donc ajouté une base SQLite (fichier campus_event.db), qui se remplit automatiquement au premier lancement à partir de ces listes. Le statut initial d'une inscription est pending. Si l'événement est complet, nous ne refusons plus la demande : elle est enregistrée avec le statut waitlisted. Quand une place se libère parce qu'une inscription est refusée, la première personne de la liste d'attente passe automatiquement en pending.

Choix REST : les statistiques ne sont pas une ressource à part, elles sont calculées à partir des événements et des inscriptions. Le frontend ne communique qu'avec l'API et n'accède jamais directement aux données.

Frontend : le sujet demandait au minimum d'afficher les événements récupérés via GET /events avec fetch(). C'est ce que fait la page index.html. Pour chaque événement, on voit le titre, la capacité et le nombre de places restantes. Si le serveur ne répond pas, un message l'indique à l'utilisateur.

On est ensuite allés plus loin avec quelques bonus. On a ajouté un formulaire d'inscription qui envoie un POST /registrations. Quand l'API refuse une demande (erreur 400, 404 ou 409), un message explique clairement le problème. Enfin, une vue administration permet d'accepter ou de refuser les inscriptions avec PATCH.

Pour passer de la vue étudiant à la vue administration, on a mis un simple bouton, sans système de connexion. Le sujet ne demandait pas d'authentification, donc on l'a laissée de côté. Ça reste une amélioration possible pour la suite.
Tests avec Postman : nos premières requêtes depuis Postman en ligne renvoyaient une erreur, car la version web passe par les serveurs de Postman, qui ne peuvent pas joindre notre serveur local (127.0.0.1). Il a fallu installer le Postman Desktop Agent pour que les requêtes partent de notre ordinateur. Nous avons aussi compris que l'ordre des tests comptait : une inscription créée par un test change le résultat des suivants. Avant chaque campagne de tests, nous supprimons donc le fichier campus_event.db et relançons le serveur pour repartir des données de départ.

Conclusion

CAMPUS EVENT centralise les événements, les étudiants et les inscriptions derrière une API REST Flask organisée en trois niveaux. Elle applique automatiquement les règles métier (champs obligatoires, existence, double inscription, capacité) et renvoie des codes HTTP explicites, ce qui répond aux difficultés initiales : incohérences, doubles inscriptions et manque de visibilité sur les places restantes. Nous avons aussi ajouté les deux extensions proposées dans le sujet : une base SQLite et une liste d'attente (statut waitlisted). Le système pourrait encore évoluer avec une authentification des utilisateurs.